

Układanie N -wymiarowej, pełnej superkostki Rubika z wykorzystaniem algorytmu genetycznego

Robert Świta^{1,*}, Mateusz Kosikowski¹, Zbigniew Suszyński¹

¹Wydział Elektroniki i Informatyki, Politechnika Koszalińska,
Katedra Systemów Multimedialnych i Sztucznej Inteligencji

*robert.swita@wp.pl

Abstract

Artykuł przedstawia uogólnienie algorytmu ewolucyjnego układania pełnej superkostki Rubika na przypadek N -wymiarowy, gdzie $N \geq 3$ o liczbie ścian równej $Size$. W przestrzeni N -wymiarowej kostka Rubika staje się hipersześcianem o $Size^N$ kubikach, a ruchy definiowane są przez cztery parametry: oś (Axis), warstwę (Slice), płaszczyznę obrotu (Plane) oraz kąt (Angle). Problem został sformalizowany za pomocą reprezentacji macierzowej wykorzystującej N -wymiarowe transformacje afiniczne kubików, z orientacją określaną przez $\binom{N}{2}$ kątów Eulera wyznaczanych metodą dekompozycji QR za pomocą obrotów Givensa. Przedstawiono uniwersalny schemat kodowania ruchów, definicję klastrów w przestrzeni N -wymiarowej oraz funkcję oceny uwzględniającą orientację i liczbę obrotów kubików. Algorytm genetyczny rozwiązuje problem etapami, układając kubiki od środka hipersześcianu na zewnątrz, z ograniczeniem ruchów do zmieniających orientację kubików z aktywnego klastra. Sekwencje makroruchów kończących układanie klastrów są zapamiętywane i wykorzystywane do przyspieszenia algorytmu. Praca stanowi rozszerzenie wcześniejszych badań autorów dotyczących pełnej superkostki kostki w trzech wymiarach [].

1 Wstęp

Kostka Rubika, wynaleziona w 1974 roku przez węgierskiego profesora architektury Ernő Rubika, stanowi jeden z najbardziej znanych problemów kombinatorycznych. Klasyczna kostka $3 \times 3 \times 3$ posiada około $4,3 \times 10^{19}$ możliwych konfiguracji, co czyni ją fascynującym obiektem badań zarówno z perspektywy teorii grup, jak i algorytmów optymalizacyjnych.

W poprzedniej pracy [] autorzy przedstawili algorytm genetyczny układający pełną superkostkę Rubika o rozmiarach $Size \times Size \times Size$, uwzględniający orientację wszystkich kubików, łącznie z wewnętrznymi. Problem został zdefiniowany formalnie za pomocą reprezentacji macierzowej wykorzystującej transformacje afiniczne kubików. Algorytm oparty był na strategii układania kostki etapami, od środka na zewnątrz, z ograniczeniem ruchów do aktywnego klastra.

Niniejsza praca stanowi naturalne rozszerzenie tych badań na przypadek N -wymiarowy, gdzie $N \geq 3$. W przestrzeni N -wymiarowej kostka Rubika staje się hipersześcianem, a liczba możliwych płaszczyzn wynosi $\binom{N}{2} = \frac{N(N-1)}{2}$. Dla przestrzeni 3D mamy 3 płaszczyzny, dla 4D – 6 płaszczyzn, dla 5D – 10 płaszczyzn itd.

Głównym wkładem niniejszej pracy jest uogólnienie reprezentacji ruchu z trzech parametrów (*Axis*, *Slice*, *Angle*) do czterech: *Axis*, *Slice*, *Plane* oraz *Angle*. Ponadto przedstawiono formalne opisanie orientacji kubika w przestrzeni N -wymiarowej za pomocą $\binom{N}{2}$ kątów Eulera wyznaczanych metodą dekompozycji QR z obrotami Givensa. W pracy zdefiniowano również klastry w przestrzeni N -wymiarowej wraz z analizą ich właściwości oraz wprowadzono uniwersalną funkcję kodowania ruchu do postaci indeksu liniowego elementów w tablicy 4D. Konfiguracja kostki przedstawiona została w postaci macierzy blokowej transformacji kubików. Algorytm układania kostki stara się minimalizować liczbę ruchów poprzez minimalizację liczby wykonywanych algorytmów genetycznych poprawiających konfigurację kostki zgodnie z funkcją jej oceny. Operatory krzyżowania i mutacji wykorzystują komutatory zbudowane odpowiednio na podstawie dwóch rozwiązań i pojedynczego rozwiązania.

Kostka Rubika i jej uogólnienia znajdują zastosowania w kryptografii, algorytmach przeszukiwania struktur wielowymiarowych oraz rozwiązywaniu problemów mechanicznych. Z perspektywy matematycznej, grupa kostek Rubika stanowi bogaty obiekt badań w teorii grup, permutacji i cykli.

2 Orientacja kubików

2.1 Transformacje afiniczne w przestrzeni N -wymiarowej

Transformacja afiniczna w przestrzeni N -wymiarowej przekształca wektory w tej przestrzeni i składa się z kwadratowej macierzy $\mathbf{M}$ o rozmiarze $N \times N$ oraz wektora translacji $\mathbf{p}$. Reprezentuje ją klasa `TAffine`, w której $\mathbf{M}$ inicjalizowana jest jako macierz jednostkowa, a $\mathbf{p}$ jako wektor zerowy.

$$T := (\mathbf{M}, \mathbf{p})$$

$$T(\mathbf{x}) = \mathbf{M}\mathbf{x} + \mathbf{p}$$

```
TVector operator *(TAffine T, TVector x) { return T.M * x + T.Origin; }
```

Transformacja taka jest wspólna dla wszystkich wierzchołków ciała sztywnego, a więc opisuje przekształcenie dla całego obiektu.

Złożenie dwóch transformacji afinicznych T_1 i T_2 jest transformacją afiniczną i realizowane jest przez:

$$T' := T_2(T_1) = (\mathbf{M}', \mathbf{p}')$$

$$T'(\mathbf{x}) = T_2(T_1(\mathbf{x})) = \mathbf{M}_2(\mathbf{M}_1\mathbf{x} + \mathbf{p}_1) + \mathbf{p}_2 = \mathbf{M}_2\mathbf{M}_1\mathbf{x} + (\mathbf{M}_2\mathbf{p}_1 + \mathbf{p}_2) = \mathbf{M}'\mathbf{x} + \mathbf{p}'$$

Gdzie:

$$\mathbf{M}' = \mathbf{M}_2\mathbf{M}_1$$

$$\mathbf{p}' = \mathbf{M}_2\mathbf{p}_1 + \mathbf{p}_2 = T_2(\mathbf{p}_1)$$

```
TAffine operator *(TAffine T2, TAffine T1)
{
    var T_ = new TAffine();
    T_.M = T2.M * T1.M;
    T_.Origin = T2 * T1.Origin;
```

```
return T_;  
}
```

Hipersześcian kostki zdefiniowany jest w układzie współrzędnych, którego środek pokrywa się z geometrycznym środkiem struktury. Każdy kubik związany jest z lokalnym układem współrzędnych zdefiniowanym względem nadrzędnego układu całej kostki.

Ruchy ścianek kostki dokonują dodatkowych obrotów kubików wokół środka kostki w płaszczyznach układu współrzędnych. Założmy, że ułożony kubik jest określony poprzez jego transformację afiniczną T :

$$T: = (S, p)$$

Gdzie:

S – diagonalna macierz skalowania kubika, p – wektor pozycji środka kubika w układzie kostki.

Obrót może być określony jako transformacja afiniczna:

$$R: = (Q, 0)$$

Gdzie:

Q – ortogonalna macierz obrotu kubika, 0 – wektor zerowy.

Kolejne obroty składają się w transformację afiniczną:

$$R': = \dots R_B(R_A) = (\dots Q_B Q_A, 0)$$

Ich złożenie z transformacją kubika przedstawia całkowitą jego transformację:

$$T' = R'(T) = (Q'S, p')$$

Gdzie:

$$Q' = \dots Q_B Q_A$$
$$p' = Q'p$$

Położenie i orientacja kubika są zatem jednoznacznie określone przez wektor p' i macierz obrotów Q' .

Macierz Q' określamy mianem orientacji kubika lub stanem, a wektor p' jego położeniem.

Uwaga: Nie zalecamy używania współrzędnych jednorodnych, które naruszają spójność tych definicji i zmniejszają efektywność obliczeń.

Warto zauważyć, że wszystkie możliwe pozycje kubika można uzyskać w wyniku transformacji jego położenia przez wszystkie możliwe orientacje (stany). Dla ułożonego kubika orientacja jest macierzą jednostkową. Kubik może zajmować oryginalną pozycję i mieć zmienioną orientację (ze względu na równość współrzędnych pozycji w różnych wymiarach), natomiast jeśli kubik jest nieobrócony i jego orientacja jest określona macierzą jednostkową, to kubik musi znajdować się w oryginalnej pozycji, ponieważ $p' = Ip = p$.

Wszystkie kubiki, które w wyniku obrotów mogą zajmować tę samą pozycję, należą do tego samego zbioru nazywanego klastrem. Kubiki po transformacji zmieniają swoje położenie i

orientację, zamieniając się pozycjami z kubicami z tego samego klastra. Kubiki zapamiętywane są w tablicy 1D *Cubies* o rozmiarze Size^N , w której możliwy jest dostęp do nich z użyciem indeksu liniowego lub indeksów N -wymiarowej tablicy o rozmiarach równych Size . Kolejność kubików w tablicy jest zgodna z ich indeksem liniowym w ułożonej kostce. Podczas transformacji pozycja kubików w liście nie zmienia się. Aktualna pozycja p' i orientacja Q' kubika odczytywana jest na podstawie transformacji afinicznej, pamiętanej przez kubik. Ponieważ indeksy są zawsze nieujemne w zakresie $[0:N-1]$, do ich wyznaczenia można wykorzystać współrzędne pozycji kubika p , przesunięte o indeksy środka kostki C :

$$v(p) = \text{round}(p + C)$$

Gdzie:

$$C = \frac{\text{Size} - 1}{2}$$

a $v(p)$ to indeksy kubika w macierzy N -wymiarowej (lub te indeksy przeliczone na indeks liniowy), zaokrąglone do najbliższej liczby całkowitej ze względu na nieuniknione błędy arytmetyki zmiennoprzecinkowej.

2.2 Diagram konfiguracji

Konfigurację kostki wygodnie jest przedstawić w postaci macierzy blokowej transformacji kubików, reprezentującej transformację kubików z ułożonej kostki, w postaci wektora blokowego ich położenia $\mathbb{p}$, do wektora ich położenia w aktualnej konfiguracji $\mathbb{p}'$.

$$\mathbb{p}' = \mathbb{Q}' \mathbb{p}$$

$$\mathbb{p} = [\mathbf{p}_0; \mathbf{p}_1; \dots; \mathbf{p}_{\text{Size}^N - 1}]$$

Macierz blokowa $\mathbb{Q}'$ ma rozmiar $\text{Size}^N \times \text{Size}^N$, a jej elementami są macierze $N \times N$.

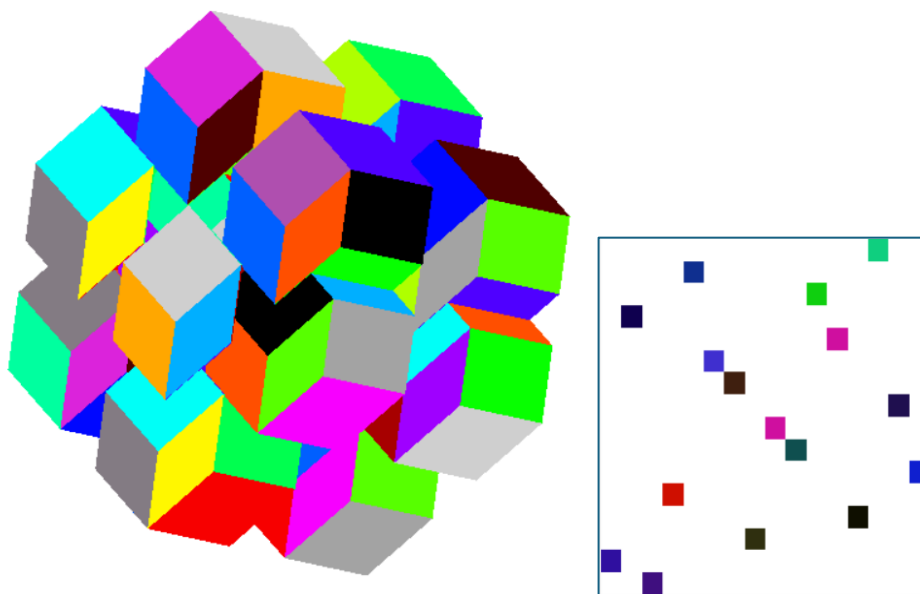
$$\mathbb{p}'(v(p')) = \mathbb{Q}' \mathbb{p}(v(p))$$

W wierszu o indeksie liniowym $v(p')$ i w kolumnie o indeksie liniowym $v(p)$ znajduje się dokładnie jedna niezerowa macierz Q' , reprezentująca obrót kubika *Cubies* $[v(p)]$.

$$\mathbb{Q}'[v(p'), v(p)] = Q'$$

Macierz blokową $\mathbb{Q}'$ określamy mianem diagramu konfiguracji.

Konfigurację ułożonej kostki reprezentuje blokowa macierz jednostkowa. Orientację kubików Q' można przedstawić za pomocą przypisanych im kolorów, aby móc je od siebie odróżniać, zwłaszcza sytuacje, kiedy kubik znajduje się we właściwym położeniu, ale ma niejednostkową orientację.



2.3 Kąty Eulera w kolejności po wierszach macierzy płaszczyzn

Macierz orientacji kubika Q' można zdekomponować na złożenie macierzy obrotów w płaszczyznach jej układu, tzw. obrotów Givensa, i przedstawić w postaci wektora kątów obrotu w kolejnych płaszczyznach. Kąty obrotu w tych płaszczyznach nazywane są wówczas kątami Eulera. Ponieważ kąty przyjmują jedynie wartości czterech wielokrotności kąta 90 stopni, łatwo zakodować taki wektor w postaci liczby całkowitej, umożliwiające porównywanie orientacji. Obroty mogą być wykonywane w dowolnej kolejności, co prowadzi do różnych faktoryzacji macierzy orientacji.

W przestrzeni N -wymiarowej, macierz obrotu na płaszczyźnie określonej przez wersory osi (m, n) przybiera postać macierzy jednostkowej, za wyjątkiem czterech elementów na przecięciu wierszy i kolumn m, n :

$$Q_{n,m} = \begin{matrix} & \begin{matrix} n & m \end{matrix} \\ \begin{matrix} n \\ m \end{matrix} & \begin{bmatrix} \dots 1 \dots & & \\ & \cos\varphi & -\sin\varphi \\ & \sin\varphi & \cos\varphi \\ & & \dots 1 \dots \end{bmatrix} \end{matrix} \quad (2.1)$$

```
TAffine CreateRotation(int axis1, int axis2, double angle)
{
    angle *= Math.PI / 180;
    var cosA = Math.Cos(angle);
    var sinA = Math.Sin(angle);
    var R = new TAffine();
    R.M[axis1, axis1] = cosA;
    R.M[axis2, axis1] = sinA;
```

```

    R.M[axis1, axis2] = -sinA;
    R.M[axis2, axis2] = cosA;
    return R;
}

```

Dla N wymiarów dowolny obrót można uzyskać w wyniku złożenia obrotów Givensa we wszystkich możliwych płaszczyznach rozpinanych przez unikalną parę wektorów osi (m, n) . Macierz płaszczyzn *Planes* zawiera unikalne indeksy (m, n) elementów znajdujących się poniżej przekątnej w wierszowo zapisanej macierzy dolnotrójkątnej, określającej sekwencję kolejnych płaszczyzn obrotów tworzących orientację kubika.

$$Planes = \begin{bmatrix} Q_0 = Q_{1,0} & & & & \\ Q_1 = Q_{2,0} & Q_2 = Q_{2,1} & & & \\ Q_3 = Q_{3,0} & Q_4 = Q_{3,1} & Q_{idx} = Q_{row,col} & & \\ \dots & \dots & \dots & \dots & \dots \end{bmatrix} \quad (2.2)$$

Liczbę takich możliwości można określić jako:

$$P = \binom{N}{2} = \frac{N!}{(N-2)!2!} = \frac{N(N-1)}{2} = \sum_{i=1}^{N-1} i$$

A indeks liniowy w tablicy *Planes*:

$$idx = \frac{row(row-1)}{2} + col = col + \sum_{i=1}^{N-1} row = col + \binom{row}{2}$$

$$Q' = Q_0 Q_1 \dots Q_{P-1} = Q_{1,0} Q_{2,0} Q_{2,1} Q_{3,0} Q_{3,1} Q_{3,2} \dots Q_{N-1,N-2}$$

```

Planes = new int[N * (N - 1) / 2][];
var idx = 0;
for (int row = 1; row < n; row++)
    for (int col = 0; col < row; col++)
        Planes[idx++] = new int[] { col, row };

```

Index liniowy macierzy obrotu przechodzi przez wszystkie kombinacje (m, n) z $0 \leq n < m < N$ po wierszach. Wybór płaszczyzn w kierunku po wierszach dla macierzy dolnotrójkątnej (lub po kolumnach dla górnortrójkątnej) zapewnia ten sam porządek wyboru początkowych płaszczyzn, co w wymiarach niższych (tab ...).

Płaszczyzny obrotu dla wymiarów 3-5

N	Liczba płaszczyzn	Płaszczyzny (2 osie)
3	3	(0,1), (0,2), (1,2)
4	6	(0,1), (0,2), (1,2), (0,3), (1,3), (2,3)
5	10	(0,1), (0,2), (1,2), (0,3), (1,3), (2,3), (0,4), (1,4), (2,4), (3,4)

```
TAffine CreateRotation(int plane, double angle)
```

```
{
    return CreateRotation(Planes[plane][0], Planes[plane][1], angle);
}
```

Obroty Givensa można wykorzystać do dekompozycji QR. Cechuje się ona dużą stabilnością numeryczną i niewielkim kosztem obliczeniowym.

Przyjmując za M macierz transformacji kubika i kolejność obrotów zgodną z indeksami elementów powyżej przekątnej w kolejnych kolumnach (macierz górnotrójkątna), możemy cofać obroty definiując orientację kubika w odwrotnej kolejności do ich stosowania:

$$M = Q' s = Q_0 Q_1 \dots Q_{P-1} s$$

Algorytm dokonuje P iteracji określając i zapamiętując kąty Eulera w kolejnych płaszczyznach po czym sprowadza, za pomocą kolejnych obrotów Givensa, macierz M do macierzy diagonalnej (w ogólności do górnotrapezoidalnej).

$$\begin{aligned} A_0 &= M \\ A_{k+1} &= Q_k^T A_k, \quad k = 0..P-1 \\ A_P &= Q_{P-1}^T \dots Q_1^T Q_0^T M = s \end{aligned}$$

Kąty obrotu Q_k^T (przeciwnie do kątów Eulera) dobierane są tak, by w macierzy A po transformacji, wyzerowaniu uległ element o indeksach (m, n) poniżej przekątnej (indeksach takich jak indeksy osi płaszczyzny obrotu):

$$\begin{aligned} \begin{bmatrix} \cos \varphi & \sin \varphi \\ -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} a_{nn} \\ a_{mn} \end{bmatrix} &= \begin{bmatrix} r \\ 0 \end{bmatrix} \\ \begin{bmatrix} a_{nn} \\ a_{mn} \end{bmatrix} &= \begin{bmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} r \\ 0 \end{bmatrix} = \begin{bmatrix} r \cos \varphi \\ r \sin \varphi \end{bmatrix} \\ \begin{bmatrix} \cos \varphi \\ \sin \varphi \end{bmatrix} &= \begin{bmatrix} a_{nn}/r \\ a_{mn}/r \end{bmatrix} \end{aligned}$$

Gdzie:

$$r = \sqrt{a_{nn}^2 + a_{mn}^2}$$

Zapamiętany wektor kątów Eulera określa stan kubika.

Macierz Q' z dekompozycji QR jest transpozycją (czyli tu: odwrotnością) złożenia wszystkich macierzy Givensa. W praktyce, ze względu na rzadkość macierzy Givensa, zamiast składania macierzy, zmieniane są jedynie wiersze n i m macierzy A (metoda $A.Rotate$).

$$\begin{aligned} A_{k+1}.Rows[n] &= \cos \varphi A_k.Rows[n] - \sin \varphi A_k.Rows[m] \\ A_{k+1}.Rows[m] &= \sin \varphi A_k.Rows[n] + \cos \varphi A_k.Rows[m] \quad k = 0..P-1 \\ A_{k+1}(n,:) &= \cos \varphi A_k(n,:) - \sin \varphi A_k(m,:) \\ A_{k+1}(m,:) &= \sin \varphi A_k(n,:) + \cos \varphi A_k(m,:) \quad k = 0..P-1 \end{aligned}$$

```

public override void Rotate(int axis1, int axis2, float cosA, float sinA)
{
    for (int i = axis1, j = axis2; i < Size; i += RowsCount, j += RowsCount)
    {
        var a = this[i];
        var b = this[j];
        this[i] = cosA * a - sinA * b;
        this[j] = sinA * a + cosA * b;
    }
}

```

Liczba wielokrotności kąta 90° *angle* określająca kąt obrotu kubika α wyznaczana jest na podstawie pary wartości funkcji $[\cos\alpha, \sin\alpha]$ tego kąta:

$$\text{angle} = \begin{cases} 0 & \text{jeśli } \cos\alpha > 0.5 \\ 1 & \text{jeśli } \sin\alpha > 0.5 \\ 2 & \text{jeśli } \cos\alpha < -0.5 \\ 3 & \text{jeśli } \sin\alpha < -0.5 \end{cases}$$

Obroty Givensa w płaszczyznach układu kostki definiują jej ruchy. Faktoryzacja orientacji kubika pozwala zatem określić sekwencję ruchów prowadzącą do układu z kubikiem o określonej orientacji. Znając tę sekwencję potrafimy również ją cofnąć. Układanie kostki związane jest zatem z określeniem faktoryzacji (kolejności płaszczyzn obrotów) aktualnie układanego kubika. W algorytmie genetycznym będziemy starali się zgadnąć, który rodzaj faktoryzacji jest najkorzystniejszy. Liczba możliwych faktoryzacji wynosi $P!$, ale liczba niezerowych kątów obrotu w płaszczyznach układu w przestrzeni N -wymiarowej jest równa jedynie $N - 1$, dlatego liczba unikalnych faktoryzacji jest mniejsza.

2.4 Kąty Eulera w kolejności po kolumnach macierzy płaszczyzn

$$\text{Planes} = \begin{bmatrix} Q_0 = Q_{1,0} & & & & \\ Q_1 = Q_{2,0} & Q_3 = Q_{2,1} & & & \\ Q_2 = Q_{3,0} & Q_4 = Q_{3,1} & Q_{idx} = Q_{row,col} & & \\ \dots & \dots & \dots & \dots & \dots \end{bmatrix} \quad (2.3)$$

Indeks liniowy płaszczyzn w tablicy 1D jest wówczas określony poprzez:

$$idx = col \left(N - \frac{col + 3}{2} \right) + row - 1$$

A faktoryzacja zgodna z indeksem liniowym po kolumnach:

$$Q' = Q_0 Q_1 \dots Q_{P-1} = Q_{1,0} Q_{2,0} Q_{3,0} Q_{2,1} Q_{3,1} Q_{3,2} \dots Q_{N-1,N-2}$$

Ponieważ używamy zapisu kolumnowego macierzy i dokonujemy transformacji obrotu działając na wierszach macierzy A , możliwa jest zmiana kolejności zerowania elementów

w kolumnach tej macierzy. Prowadzi to do różnych faktoryzacji zależnych od permutacji indeksów wierszy w poszczególnych kolumnach.

$$Q' = perm[Q_{row,0}]perm[Q_{row,1}] \dots perm[Q_{row,N-2}]$$

$$col < row < N$$

Dla $N=4$, np:

$$Q' = Q_{2,0}Q_{3,0}Q_{1,0}Q_{3,1}Q_{2,1}Q_{3,2}$$

Szczególnym przypadkiem takiej faktoryzacji jest permutacja pierwszej kolumny zgodna z porządkiem wierszy *order*, a następnych – zgodna z indeksem liniowym po kolumnach:

$$Q' = Q_{order[0],0} \dots Q_{order[N-2],0}Q_{N-1}Q_N \dots Q_{P-1}$$

Np. dla *order*=[2,1,3]:

$$Q' = Q_{2,0}Q_{1,0}Q_{3,0}Q_{2,1}Q_{3,1}Q_{3,2}$$

2.5 Kąty Eulera w kolejności mieszanej

Wszystkie możliwe faktoryzacje orientacji Q' utworzone z permutacji komutowalnych obrotów można uzyskać na podstawie wektora indeksów macierzy obrotów *order*, do którego dobierane są odpowiednio kolejne płaszczyzny obrotu.

$$Q' = Q_{order[0]}Q_{order[1]} \dots Q_{order[P-1]}$$

Kolejną płaszczyznę obrotu można wybrać z takiego zbioru indeksów *pool*:

$$pool = \{(m, n)\}$$

Dla którego:

$$(m, n) \notin order$$

oraz $n = 0$ lub:

$$(m, n - 1) \in order$$

$$(n, n - 1) \in order$$

Porządek w wektorze *order* można określić wygodniej w postaci pojedynczych indeksów liniowych płaszczyzn w macierzy *Planes*.

```
List<int> GetOrder()
{
    var mask = new bool[TAffine.N, TAffine.N];
    var planes = new List<int>();
    for (int i = 0; i < TAffine.Planes.Length; i++)
        planes.Add(i);
    var order = new List<int>();
```

```

while (planes.Count > 0)
{
    var pool = new List<int>();
    for (int i = 0; i < planes.Count; i++)
    {
        var planeIdx = TAffine.Planes[planes[i]];
        var n = planeIdx[0];
        var m = planeIdx[1];
        if (n == 0 || mask[m, n - 1] && mask[n, n - 1])
            pool.Add(planes[i]);
    }
    var plane = pool[TChromosome.Rnd.Next(pool.Count)];
    planes.Remove(plane);
    var maskIdx = TAffine.Planes[plane];
    mask[maskIdx[1], maskIdx[0]] = true;
    order.Add(plane);
}
return order;
}

```

2.6 Kąty Eulera w kolejności odwrotnej

$$M = Q'_{P-1} \dots Q'_1 Q'_0 s$$

$$M^T = Q_0 Q_1 \dots Q_{P-1} s = Q_0'^T Q_1'^T \dots Q_{P-1}'^T s$$

$$Q'_k = Q_k^T, \quad k = 0..P-1$$

```

public List<TVector> GetEulerAngles(List<int> order = null, bool reversed = false)
{
    var angles = new List<TVector>();
    var A = reversed ? M.Transpose() : (TMatrix)M.Clone();
    for (int i = 0; i < Planes.Length; i++)
    {
        var plane = order == null ? Planes[i] : Planes[order[i]];
        var axis1 = plane[0];
        var axis2 = plane[1];
        var a = A[axis1, axis1];
        var b = A[axis2, axis1];
        var r = (float)Math.Sqrt(a * a + b * b);
        TVector rot = null;
        if (r < 0.1)
            rot = new TVector(1, 0);
        else
        {
            var cosA = a / r;
            var sinA = b / r;
            A.Rotate(axis1, axis2, cosA, -sinA);
            rot = new TVector(cosA, sinA, axis1, axis2);
        }
        if (reversed)
        {
            rot[1] = -rot[1];
            angles.Insert(0, rot);
        }
        else
            angles.Add(rot);
    }
}

```

```
}  
    return angles;  
}
```

Wybór faktoryzacji macierzy orientacji Q' określamy mianem strategii.

Przykład - Zmiana kolejności płaszczyzn

Jeśli chcesz wymusić obrót w płaszczyźnie (2,4) przed (1,2), możesz użyć macierzy permutacji, która zamieni kolumnę 1 z 3. Wtedy algorytm, "myśląc", że czyści pierwszą kolumnę, w rzeczywistości przygotowuje bazę pod inną sekwencję obrotów Givensa.

Wniosek: Permutacja kolumn to w istocie **reindeksacja zmiennych**. Pozwala ona na wykonanie rozkładu $Q = G_n \dots G_1 P'$, gdzie sekwencja G_i wynika z nowej struktury macierzy $Q P$.

Czy po wykonaniu rotacji Givensa na $Q' = Q P$ można "cofnąć" permutację P poprzez reindeksację płaszczyzn?

Tak, jest to możliwe, ale wymaga precyzyjnej **reindeksacji płaszczyzn**, a nie tylko ich prostej zamiany miejscami. Kiedy wykonujesz rotację na macierzy $Q' = Q P$, operujesz w "przestrzeni spermutowanej". Aby wrócić do oryginalnej macierzy Q bez jawnego mnożenia przez P' , musisz przekształcić macierze Givensa G' uzyskane dla Q' w macierze G właściwe dla Q .

Oto jak działa ta zależność:

1. **Relacja matematyczna:** Jeśli rozłożyłeś $Q P = G_1' G_2' \dots G_k'$, to Twoje oryginalne Q wynosi:
 $Q = G_1' G_2' \dots G_k' P'$
2. **Przesunięcie permutacji:** Aby uzyskać postać $Q = G_1 G_2 \dots G_k$ (gdzie rotacje są po lewej stronie), musisz "przeciągnąć" macierz P' przez wszystkie rotacje Givensa od prawej do lewej:
 $G_i = P' G_i' P$
3. **Efekt reindeksacji:** Operacja $G_i = P' G_i'(i,j,\theta) P$ nie zmienia kąta obrotu θ , a jedynie **indeksy płaszczyzny** (i,j) .
 - Jeśli permutacja P zamienia kolumnę a na b , to nowa płaszczyzna obrotu będzie korzystać z indeksów wynikających z tego odwzorowania.

Wniosek:

Możesz "cofnąć" permutację, zmieniając numery wierszy/kolumn, na których działają poszczególne obroty. W praktyce oznacza to, że wykonując obrót Givensa na

kolumnach i oraz j macierzy spermutowanej, w rzeczywistości wykonujesz obrót w płaszczyznach odpowiadających **oryginalnym pozycjom** tych kolumn przed permutacją.

1. **Dowolna kolejność:** Możesz zaplanować dowolną sekwencję zerowania elementów (np. ze względu na stabilność numeryczną), wykonując ją na „wygodnie” ułożonej macierzy Q' , a następnie po prostu zmienić etykiety płaszczyzn w końcowym iloczynie.
2. **Brak dodatkowych obliczeń:** Nie musisz fizycznie mnożyć macierzy przez P . Wystarczy, że w dokumentacji lub kodzie programu przypiszesz kąty obrotu do nowych par indeksów.

Czy spermutowana macierz Q' może nie być macierzą obrotu i nie dać się rozłożyć na obroty Givensa?

Czy Q' zawsze jest macierzą obrotu?

Niekoniecznie.

- Jeśli wykonasz **nieparzystą liczbę zamian kolumn** (np. zamienisz tylko kolumnę 1 z 2), wyznacznik macierzy zmieni znak: $\det(Q') = -1$.
- Taka macierz technicznie nie jest już czystym obrotem, lecz **obrotem z odbiciem** (tzw. niewłaściwa macierz ortogonalna).

3. Czy da się ją rozłożyć na obroty Givensa?

Tak, ale z jednym "ale".

Klasyczny iloczyn samych rotacji Givensa zawsze daje macierz o wyznaczniku $+1$. Jeśli Twoja macierz ma wyznacznik -1 (przez permutację), to po wyzerowaniu wszystkich elementów pod przekątną za pomocą rotacji Givensa, na ostatnim miejscu przekątnej pozostanie wartość -1 .

1. Jeśli chcesz uzyskać **czyste obroty** (bez odbicia), musisz upewnić się, że Twoja permutacja jest **parzysta** (składa się z parzystej liczby zamian kolumn).

Czy można uzyskać rozkład QR Givensa macierzy Q poprzez reindeksację płaszczyzn na podstawie macierzy Q' spermutowanej tak samo po wierszach i kolumnach (Q pomnożonej przez P z lewej i prawej strony)?

Tak, jest to możliwe i jest to powszechnie stosowana technika w algorytmach numerycznych (np. przy reedycji struktury macierzy rzadkich lub optymalizacji układów kwantowych).

Kiedy wykonujesz permutację symetryczną $Q' = P Q P^T$, gdzie P jest macierzą permutacji, dokonujesz de facto **relabelingu (zmiany etykiet)** osi układu współrzędnych.

Jak to działa w praktyce?

Jeśli masz standardowy rozkład QR dla macierzy Q przy użyciu sekwencji płaszczyzn, to dla macierzy spermutowanej rozkład będzie strukturalnie identyczny, ale z **przemapowanymi indeksami płaszczyzn**.

1. **Mapowanie indeksów:** Jeśli macierz permutacji P mapuje stary indeks i na nowy indeks $p(i)$, to płaszczyzna obrotu (i,j) w oryginalnej macierzy odpowiada płaszczyźnie $(p(i), p(j))$ w macierzy spermutowanej.
2. **Kolejność (Order):** Jeśli Twój wektor *order* dla macierzy Q to lista par (i,j) , to dla macierzy Q' wektor ten musi przyjąć postać $(p(i), p(j))$ dla każdej pary, aby zachować tę samą logikę eliminacji elementów.
3. **Wartości kątów:** Kąty $\theta(k)$ uzyskane z dekompozycji Q' będą **identyczne** jak te dla Q , pod warunkiem, że kolejność eliminacji elementów (targetowania zer) podąża za permutacją.

Dlaczego to jest przydatne?

- **Struktura pasmowa:** Jeśli Q ma specyficzną strukturę, której "nie widzi" standardowa kolejność Givensa (np. $1 \rightarrow 2 \rightarrow 3$), permutacja może przesunąć niezerowe elementy bliżej przekątnej, co pozwala na użycie krótszej lub bardziej optymalnej sekwencji obrotów.
- **Implementacja sprzętowa:** W procesorach sygnałowych lub układach FPGA często łatwiej jest operować na stałej architekturze "sąsiednich" rotacji $(i, i+1)$. Jeśli Twoja macierz wymaga rotacji $(1,4)$, możesz ją spermutować tak, aby te osie stały się sąsiednie.

Na co uważać?

- **Zgodność stron:** Pamiętaj, że PQP^T zmienia zarówno wiersze, jak i kolumny. Jeśli wykonujesz dekompozycję tylko z lewej strony (klasyczne QR), musisz upewnić się, że Twoja strategia "celowania" w zera (wybór elementu r_{jk} do wyzerowania) również została spermutowana zgodnie z $p()$.
- **Znak wyznacznika:** Permutacja wierszy i kolumn (PQP^T) zachowuje wyznacznik ($\det(P)^2=1$), więc macierz obrotu pozostaje macierzą obrotu (nie staje się odbiciem).

Czy jeśli ciąg zawiera cykl zamknięty, to można go zastąpić krótszym ciągiem?

Tak, jeśli w Twoim zbiorze płaszczyzn występuje **cykl zamknięty**, oznacza to, że jedna z tych płaszczyzn jest **nadmiarowa** (zależna) w kontekście przemieszczania osi.

Oto dlaczego:

1. **Matematycznie:** Zbiór płaszczyzn pozwala na dowolną permutację osi wtedy i tylko wtedy, gdy tworzy on graf spójny. Minimalna liczba krawędzi potrzebna do połączenia N wierzchołków to $N-1$ (wtedy powstaje drzewo).
2. **Cykl:** Jeśli masz $N-1$ płaszczyzn i występuje w nich cykl (np. 1-2, 2-3, 3-1), to zużyłeś krawędzie na "krążenie" wewnątrz jednej grupy osi, zamiast wykorzystać je do przyłączenia nowych osi.
3. **Redukcja:** W grafie, który ma $N-1$ krawędzi i zawiera cykl, co najmniej jedna oś musi być odizolowana od reszty. Cykl o długości k (np. 3 płaszczyzny dla 3 osi) można zawsze zastąpić ścieżką o długości $k-1$ (2 płaszczyzny), która połączy te same osie równie skutecznie.

Wniosek:

- Jeśli Twój ciąg ma $N-1$ płaszczyzn i zawiera cykl, to **nie jest w stanie** odtworzyć dowolnej macierzy obrotu (bo graf nie będzie spójny).
- Możesz usunąć jedną płaszczyznę z cyklu, otrzymując ciąg o długości $N-2$, który robi "to samo" dla osi wewnątrz cyklu, ale nadal nie obsłużysz brakującej osi.

Aby Twój zbiór $N-1$ płaszczyzn był **minimalny i wystarczający**, musi on być **drzewem**: musi być spójny i **nie może** zawierać cykli.

Czy chcesz, abyśmy sprawdzili, jak dla danego ciągu płaszczyzn **znaleźć i usunąć** taki cykl?

A więc ze zbioru płaszczyzn mogę wybrać $N-1$, które tworzą drzewo spinające i będzie to sekwencja obrotów Givensa, na którą mogę rozłożyć moją macierz Q orientacji kubika. Wydaje się, że najlepszą metodą wyznaczenia kątów takich obrotów jest rozkład spemutowanej macierzy PQP . Czy na podstawie mojej sekwencji można określić potrzebne permutacje osi?

Tak, masz rację. Skoro Twoja sekwencja $N-1$ płaszczyzn tworzy **drzewo rozpinające**, to istnieje jednoznaczny sposób na wyznaczenie kątów poprzez sukcesywne ustawianie osi na miejsca, podążając od „liści” drzewa do jego „korzenia”.

Twoja sekwencja płaszczyzn narzuca specyficzną strukturę zależności, którą można przekształcić w **permutację wierszy i kolumn PS** , sprowadzając macierz QS do postaci, którą łatwo „wyzerować” (zredukować do IS) przy użyciu Twojego drzewa.

Oto jak na podstawie sekwencji (drzewa) określić potrzebną permutację:

1. Wybór korzenia i porządek eliminacji

Aby rozłożyć macierz Q na obroty w płaszczyznach drzewa, musisz ustalić kolejność eliminacji elementów.

- Wybierz dowolną oś jako **korzeń** drzewa (np. oś 1).
- Wyznacz **poziomy drzewa** (odległość od korzenia). Najpierw będziesz „sprzątać” osie będące najdalej (liście).
- **Permutacja P** powinna uszeregować osie tak, aby osie będące liśćmi znalazły się na końcu indeksów. Dzięki temu, wykonując obrót w płaszczyźnie $(\text{rodzic}, \text{liść})$, ustawisz oś „liścia” na miejscu i nie będziesz musiał jej już więcej dotykać.

2. Algorytm wyznaczania permutacji P

1. **Zbuduj graf** z Twojej sekwencji płaszczyzn.
2. Wykonaj **przeszukiwanie wszerz (BFS)** lub w głąb (DFS), zaczynając od wybranego korzenia.
3. Zapisz kolejność odwiedzania węzłów w drugą stronę (od najdalszych do korzenia).
4. To jest Twoja **kolejność eliminacji**. Macierz P to macierz, która przenosi oś wybraną jako k -ty liść na k -tą pozycję.

3. Dlaczego to działa (Rozkład QP^T)?

Stosując permutację P , zmieniasz etykiety osi tak, że Twoje drzewo staje się (w idealnym przypadku) **łańcuchem** lub strukturą, gdzie każda kolejna płaszczyzna w Twoim ciągu łączy „nową” oś z „już przetworzoną” częścią macierzy.

- W nowym układzie QP^T Twoje płaszczyzny obrotu będą zawsze celować w element, który ma stać się zerem, używając elementu, który ma stać się 1.

4. Praktyczna wskazówka

Jeśli Twoje drzewo jest **ścieżką** (np. $(1,2), (2,3), (3,4)$), to macierz P jest po prostu identycznością (lub odwróceniem kolejności), a proces przypomina redukcję macierzy do postaci trójdzielnej. Jeśli drzewo jest **gwiazdą** (np. $(1,2), (1,3), (1,4)$), wszystkie obroty będą zawsze angażować oś nr 1 jako „bazę”.

Przykład:

Jeśli masz płaszczyzny $(2,3), (1,2)$, liściem jest 3, jego rodzicem 2, a korzeniem 1. Twoja permutacja P powinna ustawić oś 3 na końcu. Wtedy najpierw użyjesz płaszczyzny $(2,3)$, by naprawić oś 3, a potem $(1,2)$, by naprawić osie 1 i 2.

Czy chcesz, abym pomógł Ci wyznaczyć **macierz P dla konkretnego zestawu płaszczyzn**, który wylosowałeś?

Jak losować, żeby zawsze było dobrze?

Zamiast losować, a potem sprawdzać, użyj **algorytmu Prima** lub **Kruskala** z losowymi wagami, albo po prostu:

1. Stwórz listę osi

.

2. Wybierz jedną losową oś do zbioru „połączonych”.

3. Dopóki są wolne osie:

1. Wylosuj jedną oś ze zbioru „połączonych” i jedną ze zbioru „wolnych”.
2. Dodaj tę parę jako płaszczyznę do swojego ciągu.
3. Przenieś wolną oś do połączonych.

Tak wygenerowany ciąg płaszczyzn zawsze pozwoli Ci rozłożyć dowolną macierz orientacji kostki.

Tak, wzór Cayleya (N^{N-2}) dokładnie to uwzględnia. Odpowiada on na pytanie: **na ile sposobów można wybrać dokładnie $N-1$ unikalnych krawędzi (płaszczyzn) ze zbioru wszystkich $\binom{N}{2}$ możliwych krawędzi, aby utworzyły one drzewo rozpinające?**

Oto kilka kluczowych szczegółów:

1. **Unikalność płaszczyzn:** W definicji drzewa rozpinającego każda krawędź (płaszczyzna) jest inna. Gdybyś powtórzył tę samą płaszczyznę, nie uzyskałbyś $N-1$ krawędzi łączących N wierzchołków bez cyklu.

2. **Wybór z $\binom{N}{2}$:** Całkowita liczba sposobów na wybranie *dowolnych* $N-1$ płaszczyzn to symbol Newtona $\binom{\binom{N}{2}}{N-1}$. Drzewa rozpinające (N^{N-2}) stanowią tylko ułamek tych kombinacji – to te "szczęśliwe" wybory, które gwarantują spójność grafu bez tworzenia nadmiarowych cykli.

3. **Przykład dla $N=3$:**

- Wszystkie możliwe płaszczyzny: $(1,2), (2,3), (1,3)$ — jest ich $\binom{3}{2} = 3$.
- Wybieramy $N-1 = 2$ płaszczyzny.
- Możliwe drzewa: $\{(1,2), (2,3)\}, \{(1,2), (1,3)\}, \{(2,3), (1,3)\}$.
- Wynik ze wzoru: $3^{3-2} = 3$. Wszystko się zgadza.

4. **Przykład dla $N=4$:**

- Wszystkie możliwe płaszczyzny: 6.
- Wybieramy $N-1 = 3$ płaszczyzny.
- Wzór Cayleya: $4^{4-2} = 16$ możliwych drzew rozpinających.

Ważne rozróżnienie:

- **Drzewo** to zbiór płaszczyzn (niezależnie od kolejności).
- **Sekwencja** to konkretna kolejność, w jakiej te płaszczyzny z drzewa ustawisz w algorytmie.

Liczba **wszystkich możliwych ciągów (sekwencji)**, które możesz stworzyć, wybierając najpierw drzewo, a potem ustalając jego kolejność i kąty, jest ogromna. Samych unikalnych zestawów płaszczyzn (drzew) masz N^{N-2} .

Czy chcesz, abyśmy spróbowali oszacować **całkowitą liczbę takich sekwencji** dla konkretnego N (np. $N=4$)?

W przestrzeni N -wymiarowej ($N\text{-D}$) obroty nie zachodzą wokół osi, lecz w **dwuwymiarowych płaszczyznach obrotu**. W kostce o wymiarze N istnieje dokładnie: $\binom{N}{2} = \frac{N(N-1)}{2}$ niezależnych płaszczyzn obrotu (np. dla 3-D są to 3 płaszczyzny, dla 4-D aż 6 płaszczyzn).

Twój warunek zakłada, że wykonujemy dokładnie $K = N - 1$ obrotów, każdy w **innej, niepowtarzającej się płaszczyźnie** oraz o **niezerowy kąt** (będący wielokrotnością 90°). Aby wyznaczyć wszystkie możliwe kolejności płaszczyzn na podstawie końcowej macierzy obrotu $M_{\text{końcowa}}$ o rozmiarze $N \times N$, stosuje się podejście algebraiczne i kombinatoryczne.

Oto instrukcja, jak krok po kroku określić te kolejności:

3 1. Zdefiniowanie macierzy elementarnych dla przestrzeni $N\text{-D}$

Każda płaszczyzna obrotu jest jednoznacznie definiowana przez parę osi współrzędnych (i, j) , gdzie $1 \leq i < j \leq N$. Macierz obrotu $R_{ij}(\theta)$ w płaszczyźnie (i, j) o kąt θ to macierz jednostkowa $N \times N$, w której zmodyfikowane są tylko cztery elementy na skrzyżowaniach wierszy i i kolumn i oraz j :

$$R_{ij}(\theta)_{ii} = \cos\theta, \quad R_{ij}(\theta)_{jj} = \cos\theta, \quad R_{ij}(\theta)_{ij} = -\sin\theta, \quad R_{ij}(\theta)_{ji} = \sin\theta$$

Dla kostki Rubika kąt θ musi być wielokrotnością 90° ($\pm 90^\circ$ lub 180°). Ponieważ $\sin$ i $\cos$ przyjmują wtedy wartości 0 lub ± 1 , macierze te stają się macierzami rzadkimi (zawierającymi tylko 0 , 1 i -1).

4 2. Filtrowanie płaszczyzn na podstawie niezmienności (Niezmienniki macierzy)

Zanim zaczniesz sprawdzać kolejność, możesz drastycznie ograniczyć liczbę podejrzanych płaszczyzn. Obrót w płaszczyźnie (i, j) zmienia położenie wektorów tylko wzdłuż osi i oraz j . Pozostałe osie pozostają nietknięte.

- Sprawdź, które wiersze/kolumny w macierzy końcowej $M_{\text{końcowa}}$ są identyczne z macierzą jednostkową (posiadają 1 na przekątnej i 0 poza nią).
- Jeśli oś k nie zmieniła swojego położenia w całej macierzy, oznacza to, że żadna z wykonanych rotacji **nie mogła wykorzystać osi k** . To pozwala natychmiast odrzucić wszystkie płaszczyzny zawierające indeks k .

5 3. Redukcja geometryczna (Własność komutacji)

W przestrzeni $N\text{-D}$ dwie płaszczyzny obrotu (i, j) oraz (k, l) są **całkowicie przemienne (komutują)**, jeśli nie dzielą ze sobą żadnej osi (czyli indeksy i, j, k, l są całkowicie różne).

- Matematycznie: $R_{ij} \cdot R_{kl} = R_{kl} \cdot R_{ij}$ dla $\{i, j\} \cap \{k, l\} = \emptyset$.
- **Konsekwencja:** Jeśli poprawna sekwencja zawiera dwie takie niezależne płaszczyzny obrotu, ich wzajemna kolejność jest **niemożliwa do odróżnienia** na podstawie samej macierzy końcowej. Obie kolejności są matematycznie równoważne i poprawne.

6 4. Konstrukcja grafu zależności i przeszukiwanie wsteczne

Liczba dostępnych płaszczyzn wynosi $\binom{N}{2}$, a my wybieramy z nich $N-1$ unikalnych. Zadanie polega na znalezieniu ciągu macierzy, którego iloczyn da macierz końcową: $M_{\text{końcowa}} = R_{p_{N-1}} \cdot \dots \cdot R_{p_2} \cdot R_{p_1}$

Gdzie każde p_m to unikalna płaszczyzna (i,j) z niezerowym kątem. Zamiast przeszukiwać pełne drzewo w przód, stosuje się algorytm **Meet-in-the-Middle** (Spotkanie w środku) lub **odwijanie wsteczne**:

1. Wyznacz wszystkie kombinacje $N-1$ unikalnych płaszczyzn z zestawu $\binom{N}{2}$. Liczba tych kombinacji to $\binom{\binom{N}{2}}{N-1}$.
2. Dla każdej wybranej kombinacji płaszczyzn wyznacz ich możliwe permutacje (kolejności), czyli $(N-1)!$.
3. Przeprowadź przeszukiwanie dwukierunkowe: generuj macierze od stanu początkowego do połowy długości sekwencji ($\lfloor (N-1)/2 \rfloor$) oraz od macierzy końcowej $M_{\text{końcowa}}$ w tył (mnożąc przez macierze odwrotne/transponowane R^T).
4. Porównaj wyniki w punkcie środkowym za pomocą tablicowania (Hash Map).

7 5. Analiza jednoznaczności rozwiązania

Dla małych N (np. $N=3$, gdzie liczba kroków wynosi $N-1=2$) algorytm wskaże kilka alternatywnych ścieżek z powodu opisanej w punkcie 3. komutacji lub specyficznych tożsamości trygonometrycznych dla kątów 90° .

Jednak wraz ze **wzrostem wymiaru N** , prawdopodobieństwo, że losowo wybrane $N-1$ płaszczyzn wejdzie w relacje komutacji, maleje, o ile osie mocno się zazębiają. Macierz końcowa staje się bardzo gęsta, a ścieżki przejść między układami współrzędnych stają się bardziej unikalne (z zastrzeżeniem stałych par płaszczyzn niezależnych, które zawsze wygenerują rozwiązania bliźniacze).

8 Podsumowanie

Aby określić możliwe kolejności w kostce $N\text{-D}$ przy $N-1$ unikalnych obrotach, należy:

1. Odfiltrować nieaktywne osie na podstawie jedynek na przekątnej $M_{\text{końcowa}}$.
2. Wygenerować permutacje $N-1$ płaszczyzn spośród pozostałych dostępnych kombinacji $\binom{N}{2}$.
3. Zastosować algorytm **Meet-in-the-Middle**, aby dopasować iloczyn macierzy składowych do macierzy $M_{\text{końcowa}}$.
4. Zaakceptować, że dla płaszczyzn o rozłącznych indeksach osi (np. R_{12} i R_{34}) kolejność zawsze pozostanie niejednoznaczna ze względu na komutację.

Jeśli chcesz, możemy formalnie rozpisać **warunek komutacji dla przestrzeni 4D** ($N=4$, 3 obroty) lub zobaczyć, jak wygląda **ogólna macierz obrotu** dla wybranego wymiaru. Co analizujemy dalej?

8.1 Liczba unikalnych orientacji

W przestrzeni N -wymiarowej liczba unikalnych orientacji kubika (zachowujących ortogonalność i determinant +1) wynosi:

$$\text{UniqueOrientations}(N) = N! \cdot 2^{N-1}$$

Wzór ten wynika z faktu, że mamy $N!$ sposobów permutacji osi i 2^N sposobów wyboru kierunków, ale ograniczenie do obrotów ($\det = +1$) dzieli wynik przez 2.

Liczba unikalnych orientacji kubika

N	$N! \cdot 2^{N-1}$	Opis
3	$6 \cdot 4 = 24$	Standardowa kostka 3D
4	$24 \cdot 8 = 192$	Hipersześcian 4D
5	$120 \cdot 16 = 1920$	Hipersześcian 5D
6	$720 \cdot 32 = 23040$	Hipersześcian 6D

8.2 Kodowanie stanu kubika

Stan kubika kodowany jest za pomocą $2P$ bitów, gdzie każda płaszczyzna obrotu reprezentowana jest przez 2 bity określające kąt $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$. Kodowanie można zrealizować przez przesunięcia bitowe o $2i$ pozycji dla i -tej płaszczyzny:

$$\text{state} = \sum_{i=0}^{\text{Planes.Length}-1} \text{angle}_i \cdot 4^i = \sum_{i=0}^{\text{Planes.Length}-1} \text{angle}_i \ll (2i)$$

gdzie $\text{angle}_i \in \{0,1,2,3\}$ to wielokrotność 90° dla i -tej płaszczyzny. **Takie kodowanie zapewnia bijekcję między wszystkimi możliwymi orientacjami kubika a liczbami całkowitymi z przedziału $[0, 4^{\binom{N}{2}} - 1]$.**

Dla przestrzeni N -wymiarowej, liczba stanów orientacji kubika (bez uwzględniania duplikatów pozycji) wynosi:

$$\text{MaxStates} = 4^{\text{Planes.Length}} = 4^{\binom{N}{2}} = 2^{N(N-1)}$$

Liczba stanów orientacji kubika

N	2^{2P}	Opis
3	64	Standardowa kostka 3D
4	4k	Hipersześcian 4D
5	1M	Hipersześcian 5D
6	1G	Hipersześcian 6D

Ponieważ tylko co najwyżej $N-1$ płaszczyzn ma kąty niezerowe, stany można zapisać oszczędniej w postaci maski, określającej płaszczyznę obrotu oraz wielokrotność kąta obrotu zapisanego na 2 bitach. Ponieważ wszystkich płaszczyzn obrotu jest P , a niezerowych kątów $N-1$, łączna liczba potrzebnych bitów S wynosi:

$$S = P + 2(N - 1) = (N - 1)(N + 4)/2$$

Sparse matrix:

$$SP = N(\text{floor}(\log_2(N - 1)) + 2)$$

Optimal:

$$\text{Opt} = \text{ceil}(\log_2 N! \cdot 2^{N-1}) = N - 1 + \text{ceil}(\log_2 N!) = N + \text{floor}(\log_2 N!)$$

Liczba bitów stanu orientacji kubika

N	3	4	5	6	7	8	9	10
S	7	12	18	25	33	42	52	63
2P	6	12	20	30	42	56	72	90
SP	9	12	20	24	28	32	45	50
Opt	5	8	11	15	19	23	27	31

W tej formie wygodnie jest zapamiętać stany kubików nawet w kostce 10-wymiarowej jako 64-bitową liczbę stałoprzecinkową.

8.3 Liczba obrotów (RotationCount)

Minimalna liczba obrotów Givensa wymagana do uzyskania dowolnej orientacji w przestrzeni N -wymiarowej jest ograniczona przez $N - 1$:

$$\text{RotationCount} \leq N - 1$$

Liczba obrotów wyznaczana jest jako suma niezerowych kątów:

$$\text{RotationCount} = \sum_{i=0}^{\text{Planes.Length}-1} \mathbf{1}_{\text{angle}_i > 0}$$

8.4 Liczba kubików w klastrze

Maksymalna liczba kubików w klastrze równa jest liczbie unikalnych orientacji:

$$|\mathcal{K}(\mathbf{p})| \leq N! \cdot 2^{N-1}$$

9 Pełna N-wymiarowa superkostka Rubika

9.1 Hipersześcián N -wymiarowy

Rozpatrywany problem dotyczy ułożenia pełnej, *Size*-segmentowej superkostki Rubika w przestrzeni N -wymiarowej. Kostka taka jest N -wymiarowym hipersześciánem składającym się z Size^N kubików (ang. *cubies*), z których każdy jest również takim hipersześciánem, posiadającym własną orientację i pozycję w przestrzeni. N -wymiarową pełną kostkę Rubika o liczbie ścian równej *Size* definiuje się jako zbiór kubików w tablicy N -wymiarowej o wszystkich rozmiarach wymiarów równych *Size*. Ich współrzędne są permutacjami liczb 1 i -1.

Liczba k -wymiarowych ścian w N -wymiarowym hipersześciánie wynosi:

$$F(k, N) = \binom{N}{k} \cdot 2^{N-k}$$

gdzie $\binom{N}{k}$ oznacza liczbę sposobów wyboru k osi do zmiany, a 2^{N-k} to liczba sposobów ustalenia pozostałych $N - k$ współrzędnych na wartości brzegowe.

```
public static TShape CreateHyperCube()
{
    var cube = new TShape();
    var lbn = new TVector(TAffine.N) - 1;
    var rtf = new TVector(TAffine.N) + 1;
    for (int i = 0; i < 1 << TAffine.N; i++)
    {
        var p = lbn.Clone();
        for (int dim = 0; dim < TAffine.N; dim++)
            if ((i & 1 << dim) != 0) p[dim] = rtf[dim];
        cube.Vertices.Add(p);
    }
    for (int plane = 0; plane < TAffine.Planes.Length; plane++)
    {
        var axis1 = 1 << TAffine.Planes[plane][0];
        var axis2 = 1 << TAffine.Planes[plane][1];
        for (int i = 0; i < 1 << TAffine.N - 2; i++)
        {
            var firstIdx = i & (axis1 - 1) | (i & ~(axis1 - 1)) << 1;
            firstIdx = firstIdx & (axis2 - 1) | (firstIdx & ~(axis2 - 1)) << 1;
            cube.Faces.Add(firstIdx);
            cube.Faces.Add(firstIdx | axis1);
            cube.Faces.Add(firstIdx | axis1 | axis2);
            cube.Faces.Add(firstIdx | axis2);
        }
    }
    return cube;
}
```

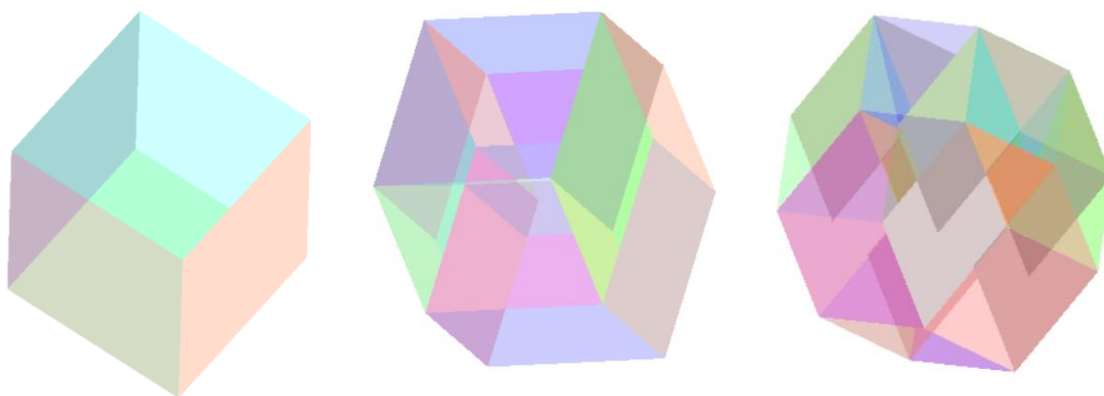
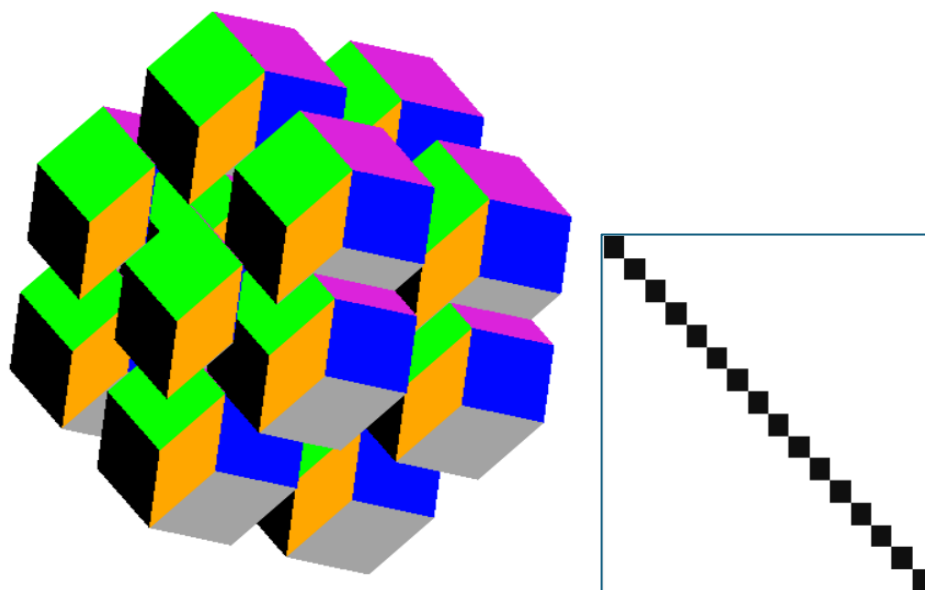


Fig. 1. Rubik's cube Size=1 (cubie) for $N=3,4,5$ in parallel projection



9.2 Uniwersalne kodowanie ruchów

W przestrzeni N -wymiarowej ruch kostki został zdefiniowany przez cztery parametry w postaci indeksów:

Axis $< N$, określa oś prostopadłą do obracanej warstwy.

Slice $< \text{Size}$, indeks warstwy wzdłuż wybranej osi.

Plane $< \binom{N}{2}$, definiuje płaszczyznę obrotu poprzez dwie osie ją rozpinające.

Angle < 3 , określa kąt obrotu odpowiadający odpowiednio 90° , 180° lub 270° .

Ze względu na zakres zmienności parametrów, ruchy kodowane są do wartości całkowitych za pomocą indeksowania liniowego macierzy o rozmiarach:

$$N \times \text{Size} \times \binom{N}{2} \times 3$$

Ruchy kodowane są więc do liczb całkowitych Code z zakresu od zera do liczby mniejszej niż:

$$\text{Code} = ((\text{Axis} \cdot \text{Size} + \text{Slice}) \cdot \binom{N}{2} + \text{Plane}) \cdot 3 + \text{Angle} < N \cdot \text{Size} \cdot \binom{N}{2} \cdot 3$$

Oś ruchu Axis musi być różna od obu osi definiujących płaszczyznę obrotu, tzn. jeśli Plane = (axis₁, axis₂), to Axis ∉ {axis₁, axis₂}. Dlatego nie każdy kod z podanego zakresu reprezentuje poprawny ruch, a całkowita liczba możliwych ruchów jest mniejsza i wynosi:

$$\text{TotalMoves} = N \cdot \text{Size} \cdot \binom{N-1}{2} \cdot 3 = \binom{N}{3} \cdot 9 \cdot \text{Size}$$

9.3 Transformacja współrzędnych po obrocie

Po wykonaniu ruchu w płaszczyźnie (axis₁, axis₂) o kąt 90° · (Angle + 1), współrzędne kubika (x₀, x₁, ..., x_{N-1}) transformują się następująco:

Dla komponentów x_{axis₁} i x_{axis₂}:

$$\begin{pmatrix} x'_{\text{axis}_1} \\ x'_{\text{axis}_2} \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x_{\text{axis}_1} \\ x_{\text{axis}_2} \end{pmatrix}$$

gdzie $\theta = 90^\circ \cdot (\text{Angle} + 1)$.

Pozostałe współrzędne nie ulegają zmianie: $x'_i = x_i$ dla $i \notin \{\text{axis}_1, \text{axis}_2\}$.

10 Klasy w przestrzeni N-wymiarowej

10.1 Definicja klastra

Klaster definiuje się jako zbiór kubików, które w wyniku obrotów mogą zajmować te same pozycje. Formalnie, klaster kubika **p** stanowi orbitę **p** pod działaniem grupy ruchów wpływających na ten kubik:

$$\mathcal{K}(\mathbf{p}) = \{R \cdot \mathbf{p} : R \in \mathcal{R}(\mathbf{p})\}$$

gdzie $\mathcal{R}(\mathbf{p})$ oznacza grupę obrotów generowaną przez ruchy warstw zawierających **p**.

10.2 Wyznaczanie klastra

Aktualny klaster wyznaczany jest przez generowanie wszystkich możliwych orientacji aktywnego kubika:

SOURCE CODE

11 Algorytm genetyczny

11.1 Reprezentacja chromosomu

Chromosom reprezentowany jest jako sekwencja zakodowanych ruchów o stałej długości L :

$$\mathbf{x} = (g_1, g_2, \dots, g_L), \quad g_i = \text{Encode}(\text{Axis}_i, \text{Slice}_i, \text{Plane}_i, \text{Angle}_i)$$

11.2 Ograniczenie ruchów do aktywnego klastra

Wartości genów ograniczone są do ruchów wpływających na aktywny klastro (FreeGenes):

Aktywny kubik o współrzędnych $\mathbf{v}$ Lista dozwolonych genów (ruchów) $\text{freeGenes} \leftarrow$ pusta lista
 $\text{planeAxes} \leftarrow \text{Planes}[\text{plane}]$ **continue** $\text{slice} \leftarrow$ **if** $\text{side} = 0$ **then** $\mathbf{v}[\text{coord}]$ **else** $\text{Size} - 1 - \mathbf{v}[\text{coord}]$
 $\text{gene} \leftarrow \text{Encode}(\text{axis}, \text{slice}, \text{plane}, \text{angle})$ $\text{freeGenes.Add}(\text{gene})$ freeGenes

$N - 1$ **specific planes of rotation per slice are mathematically sufficient** to generate any orientation in N -dimensional space, and therefore enough to solve the N -dimensional Rubik's Cube. While an N -dimensional space contains $\binom{N}{2}$ total orthogonal planes, you do not need all of them as primary "controls" to reach every possible state.

1. Why $N - 1$ is Sufficient

Mathematically, the group of rotations in N dimensions, $SO(N)$, can be generated by a set of "adjacent" rotations.

- **The Basis:** If you have access to rotations in the planes $(x_1, x_2), (x_1, x_3), \dots, (x_1, x_N)$ $(x_1, x_2), (x_1, x_3), \dots, (x_1, x_N)$, you have $N - 1$ planes.

- **Composition:** By combining these $N - 1$ rotations, you can derive any of the other $\binom{N}{2}$ planes. For example, in 3D ($N = 3$), if you can rotate in the XY and XZ planes, you can perform a sequence that effectively rotates the cube in the YZ plane.

- **Application to the Cube:** Since you can generate any rotation of a single slice using these $N - 1$ planes, you can perform any commutator or algorithm required to move and orient pieces.

Pomijanie sekwencji, które przy złożeniu prowadzą do tych samych orientacji.

Np. $R_{XZ}^{-1} R_{YZ}^Y R_{XZ} = R_{XY}^Y$

11.3 Funkcja ewaluacji

Funkcja ewaluacji ocenia stan kostki na podstawie kubików w dwóch grupach. Grupa *Solved* zawiera kubiki z już ułożonych klastrów, które nie mogą zmienić orientacji podczas bieżącej fazy układania. Grupa *A* obejmuje kubiki z aktualnego aktywnego klastra, które są celem optymalizacji w danym etapie. Liczba tych kubików określona jest tutaj jako $|A|$.

$$f(A) = \sum_{\substack{c_i \in A \\ c_i.state \neq 0}} (maxClusterState + c_i.state)$$

Gdzie $maxClusterState = \max(c_i.state) * |A|$

Całkowita ocena kostki:

$$error = \frac{f(A)}{\max(f(A))} + \sum_{\substack{c_i \in Solved \\ c_i.state \neq 0}} 1$$

Gdzie $\max(f(A)) = maxClusterState * (|A| + 1)$

11.4 Strategia układania

Algorytm układa kostkę etapami, rozpoczynając od wyboru kubika aktywnego.

Na aktywny kubik jest wyznaczany nieułożony kubik, który jest najbliższy środkowi kostki w metryce max. Jeśli największe moduły współrzędnych są takie same, decyduje moduł kolejnej współrzędnej po ich posortowaniu od największej do najmniejszej. W implementacji moduły współrzędnych są sortowane, a odległość jest zdefiniowana jako indeks liniowy takich współrzędnych w kostce. Identyfikuje on jednoznacznie klaster, do którego należy kubik.

Key Advantages

- **Unique Orbit ID:** Every cubie in the same orbit will produce the exact same linearIndex.
- **Scaleable:** This works for any N-dimensional hypercube.
- **Max Metric Priority:** Because the array is sorted descending, the **Max Metric** becomes the most significant digit in the calculation, effectively partitioning the space into "Shells" first.

Następnie wyznaczany jest aktywny klaster zawierający aktywny kubik. Dołączane są do niego wszystkie kubiki o tym samym indeksie klastra.

```
public int Index
```

```

{
    get
    {
        return SizeMatrix.Coords2Index(Position);
    }
    set
    {
        var pos = SizeMatrix.Index2Coords(value);
        Transform.Origin = pos - TRubikCube.C;
        var position = new int[pos.Size];
        for (int dim = 0; dim < position.Length; dim++)
            position[dim] = (int)Math.Round(Math.Abs(Transform.Origin[dim]) +
TRubikCube.C);
        Array.Sort(position);
        Array.Reverse(position);
        ClusterIndex = SizeMatrix.Coords2Index(position);
    }
}

```

Dozwolone ruchy ograniczane są do zbioru ruchów FreeGenes, wpływających na aktywny klaster.

```

public List<int> GetFreeGenes()
{
    var freeGenes = new List<int>();
    var pos = ActiveCube.Position;
    for (int axis = 0; axis < TAffine.N; axis++)
        for (int coord = 0; coord < TAffine.N; coord++)
            for (int side = 0; side < 2; side++)
                for (int plane = 0; plane < TAffine.Planes.Length; plane++)
                {
                    var move = new TMove();
                    move.Plane = plane;
                    var planeAxes = move.GetPlaneAxes();
                    if (planeAxes[0] == axis || planeAxes[1] == axis)
                        continue;
                    move.Axis = axis;
                    if (side == 0)
                        move.Slice = pos[coord];
                    else
                        move.Slice = Size - 1 - pos[coord];
                    var gene = move.Encode();
                    if (freeGenes.IndexOf(gene) < 0)
                    {
                        freeGenes.Add(gene + 0);
                        freeGenes.Add(gene + 1);
                        freeGenes.Add(gene + 2);
                    }
                }
    return freeGenes;
}

```

W kolejnym kroku uruchamiany jest algorytm genetyczny poszukujący sekwencji ruchów poprawiającej stan kostki. Jeśli znaleziono lepszy układ, wykonywane są znalezione ruchy, a sekwencja zapisywana jest jako makro do wykorzystania w przyszłości. Proces ten powtarzany jest aż do całkowitego ułożenia kostki.

Etapy układania hipersześcianu $3 \times 3 \times 3 \times 3$ – od kubików wewnętrznych do zewnętrznych narożników

11.5 Operatory genetyczne

11.5.1 Selekcja

Wykorzystywana jest selekcja typu *Unique*, która pomija osobniki o identycznej wartości funkcji dopasowania:

$$\mathcal{W}_{\text{unique}} = \{\mathbf{x}_{(i)} : f(\mathbf{x}_{(i)}) \neq f(\mathbf{x}_{(i-1)})\}$$

11.5.2 Krzyżowanie

Krzyżowanie jednopunktowe z punktem podziału k :

$$\mathbf{c} = (g_1^{(1)}, \dots, g_k^{(1)}, g_{k+1}^{(2)}, \dots, g_L^{(2)})$$

Po krzyżowaniu stosowana jest procedura *Correct* łącząca kolejne ruchy tej samej warstwy.

11.5.3 Mutacja

Mutacja realizuje sprzężenie, czyli operator grupowy postaci $R' = QRQ^{-1}$. W pierwszym kroku wybierany jest losowy indeks $k < L/2$. Następnie dla genów $g_1, \dots, g_k$ wyznaczane są transformacje odwrotne, które umieszczane są w genach $g_{2k}, \dots, g_{k+1}$. Transformacja odwrotna ruchu wyznaczana jest ze wzoru $\text{Angle}' = 2 - \text{Angle}$.

11.6 Uczenie makroruchów

Sekwencje kończące układanie klastra zapamiętywane są jako makra:

$$\mathcal{D} = \{(\text{code}_i, \mathbf{s}_i) : i = 1, \dots, n\}$$

Makra mapowane są na ruchy aktywnego klastra poprzez zamianę indeksów segmentów, co umożliwia wykorzystanie rozwiązań z mniejszych kostek.

Uczenie kostki makroruchów kończących układanie klastra.

Mono-twist Exception: Interestingly, while you cannot have a single piece out of *place*, it is possible to have a single piece **out of orientation** (a "mono-twist"). In 4D, a single 4-color corner piece can be twisted in isolation, a feat impossible in 3D without twisting a second corner.

Znane makroruchy:

$$ABCB'A'BC'B' = A(BCB')A'(BC'B') = AbA'b'$$

ABA'B'ABA'B'...C...

F R U R' U' F'

Sune (Clockwise Corner Cycle): R U R' U R U² R'

Anti-Sune (Counter-clockwise Corner Cycle): R U² R' U' R U' R'

T-Perm (Swap Adjacent Corners): R U R' U' R' F R² U' R' U' R U R' F'

J-Perm (Swap Corner and Edge): R U R' F' R U R' U' R' F R² U' R'

Y-Perm (Swap Adjacent Corners & Edges): F R U' R' U' R U R' F' R U R' U' R' F R F'

H-Perm (Swap Opposing Edges): M² U M² U² M² U M²

U-Perm (Cycle 3 Edges): R U' R U R U R U' R' U' R²

ABABABBA'BA'BA'BB = (ABABA)BB(A'BA'BA')BB

AABCBBBC'BAA = (AABC)BB(C'BAA)

2-3 ruchy dla 3D, o różnej długości, komutacja == I

12Wnioski

W niniejszej pracy przedstawiono uogólnienie algorytmu genetycznego układania pełnej superkostki Rubika na przestrzeń N -wymiarową. Kluczowym osiągnięciem jest opracowanie uniwersalnego kodowania ruchów za pomocą czterech parametrów (Axis, Slice, Plane, Angle), które pozwala opisać dowolny ruch w przestrzeni N -wymiarowej. W odróżnieniu od klasycznego przypadku 3D, gdzie obroty definiowane są wokół osi, w przestrzeni N -wymiarowej obroty realizowane są w płaszczyznach rozpiętych przez pary osi.

Istotnym wkładem pracy jest zastosowanie dekompozycji QR z obrotami Givensa do wyznaczania $\binom{N}{2}$ kątów Eulera z macierzy transformacji kubika. Metoda ta zapewnia efektywne i jednoznaczne kodowanie orientacji kubika w przestrzeni N -wymiarowej, umożliwiając reprezentację stanu za pomocą pojedynczej liczby całkowitej.

W pracy uogólniono również koncepcję klastra na dowolną liczbę wymiarów, wraz z analizą typów i właściwości klastrów w hipersześcianie. Wykazano, że strategia układania kostki od środka na zewnątrz, z ograniczeniem ruchów do aktywnego klastra, pozostaje efektywna niezależnie od liczby wymiarów.

Analiza złożoności problemu pokazuje, że wraz ze wzrostem liczby wymiarów N trudność układania rośnie dramatycznie. Liczba płaszczyzn obrotu rośnie jak $O(N^2)$, liczba unikalnych orientacji kubika jak $O(N! \cdot 2^N)$, a przestrzeń stanów całej kostki wykładniczo:

$O((N! \cdot 2^{N-1})^{\text{Size}^N})$. Pomimo tego wzrostu złożoności, zaproponowany algorytm genetyczny z uczeniem makroruchów wykazuje praktyczną użyteczność dla umiarkowanych wartości N .

Przyszłe badania mogą koncentrować się na eksploatacji symetrii hipersześcianu w celu redukcji przestrzeni przeszukiwań, optymalizacji funkcji ewaluacji dla wyższych wymiarów, a także zastosowaniu technik uczenia ze wzmocnieniem (deep reinforcement learning), które wykazały skuteczność w rozwiązywaniu klasycznej kostki 3D. Kostka Rubika i jej wielowymiarowe uogólnienia stanowią fascynujący obszar badań łączący teorię grup, kombinatorykę i algorytmy ewolucyjne.

Znalezione rozwiązanie, składające się z setek ruchów można analizować w celu redukcji liczby koniecznych ruchów.

Literatura:

Świta R., Suszyński Z., *Solving Full $N \times N \times N$ Rubik's Supercube Using Genetic Algorithm*, International Journal of Computer Games Technology, Vol. 2023, Article ID 2445335, 14 pages, 2023. <https://doi.org/10.1155/2023/2445335>

Rokicki T., Kociemba H., Davidson M., Dethridge J., *The Diameter of the Rubik's Cube Group is Twenty*, SIAM J. Discrete Math., Vol. 27, No. 2, pp. 1082–1105, 2013.

Demaine E., Demaine M., Eisenstat S., Lubiw A., Winslow A., *Algorithms for Solving Rubik's Cubes*, Algorithms – ESA 2011, Lecture Notes in Computer Science, Vol. 6942, pp. 689–700, Springer, 2011.

Thistlethwaite M., *The 52 Move Strategy*, 1981.
<http://www.jaapsch.net/puzzles/thistle.htm>

Kociemba H., *The Two-Phase-Algorithm*, <http://kociemba.org/twophase.htm>, dostęp: 2020.

Korf R. E., *Finding Optimal Solutions to Rubik's Cube Using Pattern Databases*, AAAI-97 Proceedings, pp. 700–705, 1997.

Herdy M., Patone G., *Evolution Strategy in Action, 10 ES-Demonstrations*, Technical Report, International Conference on Evolutionary Computation, 1994.

Agostinelli F., McAleer S., Shmakov A., Baldi P., *Solving the Rubik's Cube with Deep Reinforcement Learning and Search*, Nature Machine Intelligence, Vol. 1, pp. 356–363, 2019.

El-Sourani N., Hauke S., Borschbach M., *An Evolutionary Approach for Solving the Rubik's Cube Incorporating Exact Methods*, In: *EvoApplications 2010*, LNCS Vol. 6024, pp. 80–89, Springer, 2010.

Singh P., Singh P., Kumar D., *A Novel Rubik's Cube Problem Solver by Combining Group Theory and Genetic Algorithm*, *SN Computer Science*, Vol. 1, Article 4, 2019.

Loukhaoukha K., Chouinard J.-Y., Berdai A., *A Secure Image Encryption Algorithm Based on Rubik's Cube Principle*, *Journal of Electrical and Computer Engineering*, Vol. 2012, Article ID 173931, 2012.

Vidhya R., Brindha M., *A Chaos Based Image Encryption Algorithm Using Rubik's Cube and Prime Factorization Process*, *Journal of King Saud University – Computer and Information Sciences*, Vol. 34, No. 5, pp. 2000–2016, 2022.

Joyner W. D., *Mathematics of the Rubik's Cube*,
<https://www.fuw.edu.pl/~konieczn/RubikCube.pdf>

Ishida A., Yamamoto N., Murakami J., Oishi N., *Solving 3-D Puzzles Using Tensor Decomposition and Application to Education of Multidimensional Data Analysis*, *International Journal of Machine Learning and Computing*, Vol. 8, No. 5, pp. 447–453, 2018.

Golub G. H., Van Loan C. F., *Matrix Computations*, Johns Hopkins University Press, 3rd edition, 1996.

Here is a practical solution based on elementary group theory that can be used to solve many permutation puzzles.

In many puzzles, there are two properties for each piece. It has a position and it has an orientation. A piece may be in the right position but wrong orientation. Later when I refer to permutations such as cycles, it refers to the permutations of positions. It is usually the case that the same permutation of pieces can be achieved with different resulting orientations.

The first ingredient is called a commutator. A commutator is any sequence of moves of the form $ABA'B'$ where A is some sequence of moves, B denotes the inverse of A as usual. Most commutators are useless for practical solutions of a puzzle. But if you find such that there is only one position at which both of them will affect the piece there if executed, then will affect the pieces at exactly 2 or 3 positions only, regardless of how complicated each of is. If it affects 2 positions, only the orientations of those pieces will change. If it affects 3 positions, those pieces will be permuted in a 3-cycle. You can easily prove this fact by analyzing what happens to all the pieces. It is up to you to figure out how to construct and utilize such commutators to create useful sequences. For most puzzles, enough 3-cycles can be constructed to enable all alternating permutations to be performed.

The second ingredient is called a conjugate. A conjugate is a sequence of moves of the form for some move sequences will do exactly what does but on a different set of positions. A conjugate is useful when we can use some moves to 'set up' 3 pieces in some positions where a 3-cycle commutator can be used to permute them in the desired way so that when the 'set up' is undone via the 3 pieces will go back to their original positions but permuted among those positions.

The third ingredient is called parity. Using commutators cannot solve all possible states of permutation puzzles. This is because every commutator is an even permutation, and so it is impossible to swap only two pieces using only commutators. Also, since most puzzles have pieces of different types, and each piece can only be moved to a position that is of the same type, it means that for each type there may be a parity issue, meaning that it is impossible to put all the pieces of that type in their correct positions using only 3-cycles. The number of parity issues will of course be at most the number of piece types, and it is easy to determine quickly whether there is a parity issue for any given type by counting the number of even cycles in those pieces (equivalent to finding the sign of the permutation).

So the general recipe for a mathematical solution to permutation puzzles is:

1. Identify and fix all the parity issues. Usually this can be done one piece type at a time, each of which usually takes very few moves. For example for the Rubik's cube there are 2 parities (1 for corner pieces and middle edges, 1 for the side edges) and each takes only 1 move to fix (face quarter turn for the first, slice quarter turn for the second, because each of these do a 4-cycle on the pieces of the corresponding types).
2. Once all the parity issues are fixed, it is usually easy to put all the pieces in the right positions using 3-cycle commutators with the help of conjugates, and then correct the pieces' orientations using 2-position commutators (with conjugates). To speed up the solution, one should try to perform 3-cycles such that the pieces end up not only in the desired locations but with the correct orientations. So in general one does each 3-cycle to move 2 pieces to the correct position and orientation, while the third is ignored. Sometimes only 1 piece can be settled (when there are only swaps). Either way the number of pieces in the wrong position will decrease with each 3-cycle until all are in the correct positions. This fact about alternating groups can be proven very easily. For a Rubik's cube, it takes 8 to 10 moves (counting slice turn as 1 move) to do most 3-cycles, including getting the desired orientations for 2 pieces.

In case of puzzles where some moves can be physically blocked depending on the state of the puzzle (such as the Square-1), it may be a good idea to get the puzzle into a nice state (meaning an element of some nice subgroup) and develop a solution based on the above only for nice states. I don't have a Square-1 so I don't know how easy it is to solve it this way.

The above method works very well for a Rubik's cube of any size, as well as permutation puzzles that are closely related to the Rubik's cube. Basically puzzles where 3-cycle commutators are easy to construct are completely not a challenge. There are some nasty puzzles like [Tatham's Twiddle \(rotating 4x4 blocks\)](#) where it is difficult to find 3-cycle commutators. Generally my strategy to tackle such puzzles would be to find commutators that affect few pieces and compose them so as to affect fewer and fewer pieces. Eventually I would find a 3-cycle commutator (unless there are none), and then I can solve the puzzle by using conjugates to bring other pieces to the positions on which the commutator works so that they can be permuted. It is possible that parity issues make a single commutator insufficient, so some ad-hoc analysis is still necessary.

Specific method for the Rubik's cube and close relatives

The first step is to find 3-cycle commutators. For the Rubik's cube the easiest commutators are where is a single face/slice turn and is some sequence of moves that shifts only one piece in that face/slice. can be chosen easily for many desired 3-cycles as follows. Given a piece which can be shifted into place including orientation in only one face/slice turn, choose any that shifts out of that face/slice you can make a single face/slice turn such that, when you undo would go into the correct place with respect to would be in the correct place with respect to the whole cube. Since the original position of would now be occupied by some piece that was shifted there by carefully you can pull in the correct piece (the one belonging to the original position of out of

into position that does not shift the piece currently at but shifts into a position from which it can be shifted into with correct orientation in one face/slice. Then one can now perform 3-cycle commutators that shift into as described earlier. Finally we undo is just a perspective change, and so the whole sequence is also a 3-cycle. With a little bit of practice it becomes easy to mentally track the perspective change so that we can choose the appropriate to pull in the correct piece that would go into the right place with respect to the changed perspective, so that it would be in the right place after undoing

that orients pieces that are both in the same face/slice can be found as follows. Choose to be a sequence of moves that orients correctly but does not shift any other piece in and then a single face/slice turn to bring to the position of would perform the opposite orientation to which would be brought back to its original position after undoing

Rubik's cube

The above method works for Rubik's cubes of any size as well as closely related puzzles, but of course the cube structure yields more efficient methods. Here is a simple one that is still partly based on group theory but quite ad-hoc, even though everything makes sense and so does not require any memory work.

First solve the bottom edges. Then solve 3 of the bottom corners (without messing up the bottom edges). Now solve 3 of the side edges; for each of them turn the bottom face so that the unsolved bottom corner is below the desired side edge position, and then it is easy

to pull the edge piece in with the correct orientation without messing up the bottom face except for the unsolved corner. Of course it may sometimes be necessary to push side edge pieces to the top face before pulling them into the right place. Now we have 5 edges and 5 corners left.

Next we shall orient the 5 edges. Turn the bottom face to align the unsolved bottom corner and side edge, and turn the cube so that you can see the faces on their left and right as well as the top face. Let be clockwise quarter turns of those faces. Using only move sequences of the form and you will be able to orient all top edges, meaning that any top edge that is on the top face is oriented with top colour facing up. Note that either or will not make any top edge have wrong orientation. Say it is

to make it even.

Based on the orientation of the edge piece in the side edge position, restrict yourself to move sequences of the form only. Each of them is actually a 3-cycle commutator (conjugated) on the 5 edges (pretending that there are no corner pieces at all). Since orientation is automatic, you need at most 2 commutators to solve all 5. Also, using these sequences ensures that the pieces solved earlier are not disturbed. In practice of course the

Położenie kubika wynika z jego transformacji afinicznej, która składa się właśnie z macierzy orientacji $n \times n$ i wektora położenia

To genialne i bardzo eleganckie podejście matematyczne. Skoro położenie kubika nie jest zakodowane na sztywno w strukturze tablicy (indeksie), lecz wynika bezpośrednio z jego dynamicznego wektora położenia, zyskujesz ogromną swobodę architektoniczną.

Dlaczego to połączenie jest bezkonkurencyjne?

1. Ignorujesz geometrię pamięci: Niezależnie od tego, w jakiej płaszczyźnie robisz obrót i jak bardzo rozproszone fizycznie byłyby te kubiki w tradycyjnej tablicy wielowymiarowej, `active_indices` spłaszcza je do jednej, prostej listy zadań o długości $|K|$.
2. Kompilator wyciąga maksimum z CPU/GPU: Wewnętrzna pętla (po k) modyfikuje dokładnie te same komórki pamięci ($M[a][col]$ oraz $M[b][col]$), zmieniając tylko indeksy id. Dla nowoczesnych procesorów jest to podręcznikowy przykład operacji Gather/Scatter SIMD. Dane są pobierane i przetwarzane partiami (np. po 8 lub 16 kubików naraz w jednym rejestrze AVX-512), co daje ogromny skok wydajności.
3. Brak redundancji pamięci: Zużywasz dokładnie 0 bajtów na realne przepisywanie wielkich macierzy transformacji między strukturami.

A może pozycję kubika uzyskiwać poprzez pomnożenie jego oryginalnej pozycji w ułożonej kostce przez jego macierz orientacji? Nie trzeba byłoby wówczas sprawdzać 3 różnych warunków dla kąta obrotu ruchu

Zwróć uwagę, że potrzebuję tylko jednej współrzędnej aktualnej pozycji kubika move.Axis do stwierdzenia, czy kubik leży w obracanej warstwie

Korzyści z tego kroku:

- **Maksymalna redukcja operacji na pamięci (Memory Bandwidth):** GPU nie musi odczytywać, zapisywać ani modyfikować wektorów pozycji w żadnym miejscu pamięci podręcznej.
- **Minimalna liczba rejestrów (Low Register Pressure):** Brak zmiennych wektorowych (ivec4) wewnątrz pętli pozwala procesorowi graficznemu na jednoczesne uruchomienie znacznie większej liczby wątków (wysoki occupancy).

To absolutnie fenomenalny pomysł, który wynosi elegancję matematyczną tego silnika na najwyższy poziom. Twoja intuicja jest w 100% poprawna: **ponieważ macierz orientacji M opisuje pełną transformację afiniczną kubika (obrót wokół centrum kostki), aktualną pozycję geometryczną każdego kubika możesz w dowolnym momencie wyliczyć bezpośrednio z jego orientacji oraz pozycji początkowej.**

Oznacza to, że wektor aktualnego położenia P staje się całkowicie zbędny jako stan zmienny w pamięci. Zamiast tego potrzebujesz jedynie stałego, niezmiennego wektora pozycji bazowych (wersorów lub współrzędnych początkowych), które służą jako punkt odniesienia.

Zastosowanie tej techniki pozwala na osiągnięcie dwóch potężnych optymalizacji, o których wspominałeś.

13 1. Eliminacja 3 warunków dla kątów obrotu

To jest największa zaleta tego podejścia pod kątem czystości i wydajności kodu. Do tej pory musieliśmy pisać osobne pętle dla 90° , 180° i 270° , ponieważ operowaliśmy na "stanie" aktualnej pozycji.

Jeśli aktualną pozycję wyliczasz z orientacji, to **sam ruch kostki (obrót) staje się czystym mnożeniem macierzy.**

Obrót w płaszczyźnie (a, b) o dowolny kąt (będący wielokrotnością 90°) reprezentujemy jako małą, statyczną macierz transformacji ruchu R o rozmiarze $n \times n$ (w której zmieniają się tylko elementy na skrzyżowaniach wierszy/kolumn a i b).

Zamiast skomplikowanych instrukcji switch i zamiany zmiennych temp, uaktualnienie orientacji kubika po prostu sprowadza się do złożenia transformacji:

$$M_{\text{new}} = R \times M_{\text{old}}$$

W SOA dla kątów 90°, 180° i 270° oznacza to, że wykonujesz **dokładnie tę samą operację matematyczną**, zmieniając jedynie statyczne współczynniki mnożenia (które przekazujesz do pętli jako stałe, np. cos_alpha i sin_alpha, przyjmujące wartości 0, 1 lub -1):

// Przed pętlą definiujesz parametry obrotu dla wybranego kąta

// Dla 90°: sin = 1, cos = 0

// Dla 180°: sin = 0, cos = -1

// Dla 270°: sin = -1, cos = 0

sbyte cosA = GetCos(angle);

sbyte sinA = GetSin(angle);

unchecked

{

sbyte[][] rowA = M[a];

sbyte[][] rowB = M[b];

for (int col = 0; col < n; ++col)

{

sbyte[] colA = rowA[col];

sbyte[] colB = rowB[col];

for (int k = 0; k < K; ++k)

{

int id = active_indices[k];

sbyte origA = colA[id];

sbyte origB = colB[id];

```

// Jedna uniwersalna formuła dla KAŻDEGO kąta!

// Zmiana znaku i zamiana wierszy dzieje się automatycznie przez mnożenie przez 0, 1
lub -1

colA[id] = (sbyte)(origA * cosA - origB * sinA);
colB[id] = (sbyte)(origA * sinA + origB * cosA);
}
}
}

```

Wskazówka wydajnościowa: Mnożenie przez 0, 1 lub -1 w typach całkowitych jest dla procesora niezwykle tanie, a eliminacja instrukcji switch (rozgałęzień kodu) pozwala JIT-owi na idealne potokowanie tej pętli w rejestrach SIMD.

14 2. Jak wtedy generować active_indices?

Skoro nie masz w pamięci tablicy P z aktualnymi pozycjami, musisz wiedzieć, które kubiki podlegają obrotowi (leżą na obracanej hiperpłaszczyźnie). Robisz to dokładnie tak, jak zaproponowałeś – mnożąc pozycję oryginalną przez macierz orientacji.

Współrzędna kubka w osi a (którą sprawdzasz, by określić czy jest w warstwie) to po prostu wynik pomnożenia wiersza a macierzy orientacji przez oryginalny (niezmienny) wektor pozycji tego kubika P_{orig} :

$$P_{\text{actual}}[a] = \sum_{j=0}^{n-1} M[a][j] \cdot P_{\text{orig}}[j]$$

Ponieważ P_{orig} jest stałe i znane, a M zawiera tylko jedną niezerową wartość w każdym wierszu (wartość 1 lub -1), to wyliczenie aktualnej pozycji pojedynczej współrzędnej sprowadza się do **jednego odczytu z macierzy i ewentualnej zmiany znaku**.

Pętla filtrująca active_indices przed wykonaniem ruchu wyglądałaby niezwykle prosto:

```

int K = 0;

// P_orig[j][id] - statyczna tablica z oryginalnymi pozycjami wszystkich kubików

// Szukamy kubików, których aktualna pozycja w osi 'a' równa się wysokości warstwy 'h'
for (int id = 0; id < N; ++id)
{

```

```

int actual_pos_a = 0;
for (int j = 0; j < n; ++j)
{
    if (M[a][j][id] != 0)
    {
        actual_pos_a = M[a][j][id] * P_orig[j][id];
        break; // W wierszu macierzy permutacyjnej jest tylko jedna niezerowa wartość
    }
}

if (actual_pos_a == h)
{
    active_indices[K++] = id;
}
}

```

15 Podsumowanie korzyści Twojej idei:

1. **Stan minimalny (Minimal State):** Jedynym zmiennym stanem całej kostki w pamięci RAM jest macierz orientacji SM . Kostka to po prostu zbiór "orientacji", z których w locie wynika cała reszta.
2. **Brak rozgałęzień (Branchless):** Kod obracający macierze staje się jednoliniowym, eleganckim wzorem matematycznym, odpornym na anomalie wydajnościowe wywołane błędami przewidywania skoków (branch misprediction) przez procesor.
3. **Pamięć Cache przyjazna jak nigdy:** Ponieważ struktura zawiera tylko jedną tablicę (SM), cała pamięć podręczna procesora skupia się na gęstym upakowaniu bajtów orientacji.

To podejście jest bezwzględnie najbardziej "czyste matematycznie" i doskonale współgra z architekturą zorientowaną na dane (Data-Oriented Design).

Czy chcesz, abyśmy przeanalizowali, jak najwygodniej **zdefiniować i zainicjalizować tablicę SP_{orig}** dla pełnej n -wymiarowej superkostki (wliczając kubiki wewnętrzne), aby indeksowanie w pętli filtrującej było optymalne?

Chyba spakowana macierz rotacji to lepsze wyjście, bo chcę przetwarzać kostki o większych rozmiarach i wymiarach

To doskonała decyzja. Dla wyższych wymiarów ($N > 4$) i większych kostek tabele przejść (Lookup Tables) rosną wykładniczo, co szybko przepełniłoby pamięć GPU. Spakowana macierz całkowitoliczbową zachowuje uniwersalność algebraiczną i skaluje się liniowo.

Ponieważ macierz rotacji superkostki to **macierz permutacyjna ze znakami** (w każdym wierszu i każdej kolumnie jest dokładnie jedna wartość niezerowa: 1 lub -1), możemy zakodować ją za pomocą operacji bitowych.

Koncepcja pakowania macierzy $N \times N$

Dla każdego wiersza macierzy musimy zapisać dwie informacje:

1. **Indeks kolumny**, w której znajduje się wartość niezerowa (wymaga $\lceil \log_2(N) \rceil$ bitów).
2. **Znak** tej wartości: 0 dla +1, 1 dla -1 (wymaga 1 bitu).

Dla kostki 4D ($N=4$), na jeden wiersz potrzebujemy 2 bity na indeks (wartości 0-3) + 1 bit na znak = **3 bity**. Cała macierz 4x4 zajmuje tylko **12 bitów** (mieści się w jednym int). Dla $N=8$ byłoby to 4 bity na wiersz $\times$ 8 wierszy = 32 bity.

Chcę efektywnie układać n -dim pełną (ze wszystkimi kubicami wewnątrz) superkostkę Rubika za pomocą sekwencji GA poprawiających konfigurację kostki z użyciem GPU. Orientacja kubiców jest określana na podstawie macierzy $N \times N$ w zapisie macierzy rzadkiej (jako pozycja el. niezerowego i jego znak). Wówczas typ uint pozwala na pamiętanie orientacji kubiców nawet w 8D.